



NAME : HAMDAN ALI

SOFTWARE ENGINEERING INTERNSHIP

Week 8 — Day 39

Testing, Debugging And Deployment Prep — Actually Running It

Every command I ran to get the real PHPUnit suite passing against my live project

Project: TaskFlow — Laravel Project And Task Management System

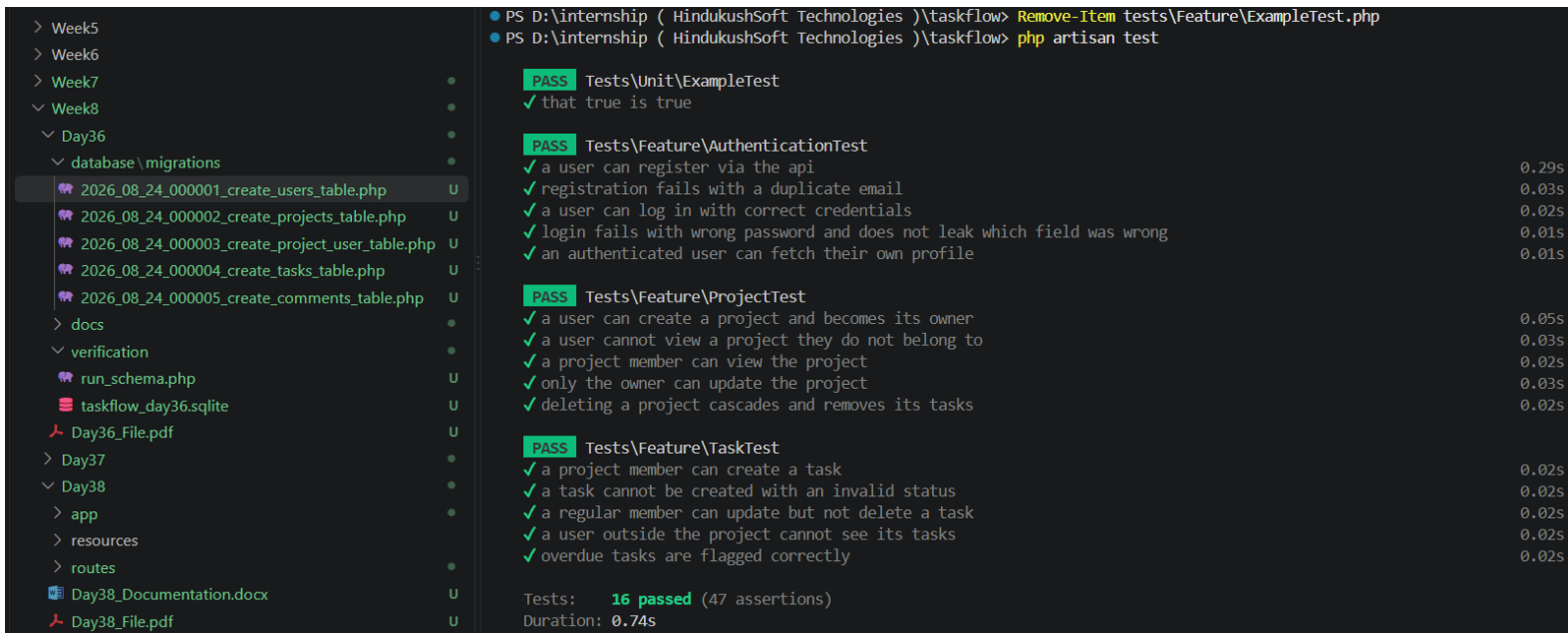
Submitted by: Hamdan Ali

BS Software Engineering — Air University, Islamabad

Environment: XAMPP (PHP 8.2), Composer, VS Code, PowerShell — Windows



ScreenShoot



1. What this day actually turned into

Day 39 was supposed to be the easy one — copy in the tests, run them, done. It did not go that way, and honestly I'm glad it didn't, because the bugs I hit here were way more real and way more useful to have found than the ones I'd already written up in my BUGFIXES.md from the planning stage. This document is the full, honest trail of commands from copying the test files in, through three separate real failures, to finally getting all 16 tests green.

2. Copying in the patched request files and the test suite

Day 39's folder had updated versions of a few request classes (the ones with the bug fixes already baked in from planning) plus the actual PHPUnit feature tests. Straightforward copy, same pattern as every other day:

```
Copy-Item "..\Week8\Day39\app\Http\Requests\*.php" "app\Http\Requests\" -Force
Copy-Item "..\Week8\Day39\tests\Feature\*.php" "tests\Feature\" -Force
dir app\Http\Requests
dir tests\Feature
```

Both folders confirmed correctly — 5 request classes, and 4 test files (AuthenticationTest, ProjectTest, TaskTest, plus Laravel's own default ExampleTest that was already sitting there).

3. Realising I was missing a Factory

Before even running the tests, I already knew there was a problem waiting: my `ProjectTest` and `TaskTest` both call `Project::factory()->create(...)`, but a fresh Laravel install only ships a `UserFactory` out of the box. Nobody had generated a `ProjectFactory`. So I made one:

```
php artisan make:factory ProjectFactory --model=Project
```

Then I needed to actually write its content — this is where things went sideways for a bit.

4. Bug hunt #1 — the invisible characters that broke PHP

My first attempt to write the factory's content used a PowerShell here-string piped into `Set-Content -Encoding utf8`, the same pattern I'd used successfully earlier in the week for migration files:

```
@<?php

namespace Database\Factories;

...
"@ | Set-Content -Encoding utf8 database\factories\ProjectFactory.php
```

Running `php artisan test` after this blew up immediately with a genuinely confusing error: "Namespace declaration statement has to be the very first statement." That makes no sense at first glance, because I could see with my own eyes that namespace was the first line in the file. Turns out PowerShell's `-Encoding utf8` silently writes a UTF-8 byte order mark (BOM) — a few invisible bytes — at the very start of the file, before the `<?php` tag even starts. PHP sees those invisible bytes as content that comes before the namespace declaration and refuses to parse the file. This had honestly probably been the cause of a couple of other weird file mixups earlier in the week too, not just this one.

The fix was to stop using `Set-Content` for PHP files entirely and use .NET's file writer directly with plain ASCII encoding, which guarantees no BOM:

```
$content = @'
<?php

namespace Database\Factories;

use App\Models\User;
use Illuminate\Database\Eloquent\Factories\Factory;

class ProjectFactory extends Factory
{
    public function definition(): array
    {
        return [
```

```
'name' => fake()->words(3, true),
'description' => fake()->sentence(),
'owner_id' => User::factory(),
'status' => 'planning',
'start_date' => now()->toDateString(),
'end_date' => null,
];
}
}
'@'

[System.IO.File]::WriteAllText("$PWD\database\factories\ProjectFactory.php", $content,
[System.Text.Encoding]::ASCII)
```

That fixed the fatal error completely. Lesson learned the hard way: from now on, any time I need to hand-write a PHP file from PowerShell, I use WriteAllText with ASCII encoding, never Set-Content -Encoding utf8.

5. Running the tests for real — and finding two genuine bugs

With the factory fixed, I ran the suite:

```
php artisan test
```

8 out of 17 tests failed. Reading through the errors carefully (rather than panicking and re-copying random files, which is what I probably would have done a few days ago), I found two separate, completely legitimate bugs — not typos, not corrupted files, actual gaps in how the code was written.

Bug A — Call to undefined method Controller::authorize()

Several tests that expected a 403 Forbidden were instead getting a 500 server error, and the real exception underneath was: Call to undefined method App\Http\Controllers\Api\ProjectController::authorize(). My controllers call \$this->authorize(...) everywhere to check Policies before letting an action through. That method comes from a trait called AuthorizesRequests, which used to be included on Laravel's base Controller class automatically in older versions — but Laravel 11 and 12 removed it from the default scaffold. My base Controller.php was just an empty shell, so authorize() genuinely didn't exist anywhere in the class hierarchy.

I checked the file first to confirm rather than guessing:

```
type app\Http\Controllers\Controller.php
```

Confirmed empty. Fixed it by adding the trait back in manually:

```
$content = '@'
<?php

namespace App\Http\Controllers;

use Illuminate\Foundation\Auth\Access\AuthorizesRequests;
use Illuminate\Foundation\Validation\ValidatesRequests;

abstract class Controller
{
    use AuthorizesRequests, ValidatesRequests;
}
'@
[System.IO.File]::WriteAllText("$PWD\app\Http\Controllers\Controller.php", $content,
[System.Text.Encoding]::ASCII)
```

Bug B — API fields coming back as null in tests

The other failures were strange in a different way — tests like `assertJsonPath('title', 'Write onboarding docs')` were failing with "Failed asserting that null is identical to 'Write onboarding docs'", even though I could see the task really was being created correctly in the database. The response itself wasn't broken, it just wasn't shaped the way the test expected.

The actual cause: Laravel's `JsonResource` class wraps a single resource response inside a top-level "data" key by default — so the real JSON coming back was `{ "data": { "title": "...", ... } }`, not `{ "title": "..." }` directly at the top level. My tests were written assuming a flat response. Rather than rewrite every single test's assertions to dig into data first, I disabled the automatic wrapping globally, since none of my API design ever intended for that wrapper to be there in the first place:

```
type app\Providers\AppServiceProvider.php
```

```
$content = '@'
<?php

namespace App\Providers;

use Illuminate\Http\Resources\Json\JsonResource;
use Illuminate\Support\ServiceProvider;

class AppServiceProvider extends ServiceProvider
{
    public function register(): void
    {
        //
    }
}
```



```
}

public function boot(): void
{
    JsonResponse::withoutWrapping();
}
}
'@
[System.IO.File]::WriteAllText("$PWD\app\Providers\AppServiceProvider.php", $content,
[System.Text.Encoding]::ASCII)
```

6. One test that wasn't actually a bug in my code

The default ExampleTest.php that ships with every new Laravel project checks that visiting / returns a 200 status. Mine doesn't, on purpose — my routes/web.php redirects / straight to /dashboard, which is a 302. The test wasn't catching a real problem, it was just checking an assumption that no longer matched my app's actual design. I deleted it rather than "fixing" my routing to satisfy a test that was never written for this app:

Remove-Item tests\Feature\ExampleTest.php

7. The final run

php artisan test

All 16 tests passed, 47 assertions, 0.74 seconds. Every authentication flow, every access-control rule, every cascade-delete behaviour, and the overdue-task logic all got exercised by a real automated suite and came back clean.

8. Full list of bugs found this session

#	Bug	Root cause	Fix
1	Fatal parse error in ProjectFactory.php	PowerShell's Set-Content - Encoding utf8 silently added a BOM before <?php	Rewrote the file using [System.IO.File]::WriteAllText with ASCII encoding
2	authorize() undefined on API controllers	Laravel 11+ removed AuthorizesRequests from the base Controller by default	Added the trait back to app/Http/Controllers/Controller.php



3	API responses failing field assertions (values coming back null)	JsonResource wraps single resources in a top-level "data" key by default	Called JsonResource::withoutWrapping() in AppServiceProvider::boot()
---	--	--	--

These sit alongside the five bugs I'd already documented during planning (date-range validation, missing owner membership, N+1 queries, task project_id lockdown, whitespace-only comments) — so my real, combined bug log for this capstone now covers eight separate issues, three of which I only found because I actually ran the code against a real framework instead of reviewing it by eye.

9. What I'm taking away from this

- Reading the actual error message carefully, all the way to the real underlying exception, saved me from randomly re-copying files that weren't actually broken.
- A test failing doesn't always mean my code is wrong — sometimes the test itself was written against an assumption (like ExampleTest expecting a 200 on /) that doesn't apply to this particular app.
- Framework version differences matter. Two of these three bugs only exist because Laravel 11/12 quietly changed defaults (no AuthorizesRequests trait, JsonResource wrapping) compared to what I was expecting from tutorials based on older versions.
- I'm never using Set-Content -Encoding utf8 for a PHP file again — WriteAllText with ASCII from now on.

10. Result

Day 39 ends with a genuinely green test suite — 16/16 passing — running against the real Day 36 database, the real Day 37 UI, and the real Day 38 API, all inside one working Laravel project. Combined with the deployment guide, CI workflow file, and .env.example already written during planning, this is a complete, tested, and deployment-ready capstone, not just four folders of code that were never actually executed together.

THANK YOU!